

SQL Server 事务隔离级别实验报告

Lecture 14

【23336128】

【梁力航】

实验信息

实验名称: 事务隔离级别实验

实验日期: 2025-12-11

数据库: School_Data

实验目的: 理解不同隔离级别对并发读写的影
响，观察脏读、不可重复读、
幻象读及其防护

实验内容概述

本次实验主要围绕 SQL Server 中的四种事务隔离级别展开，分别是 READ UNCOMMITTED、READ COMMITTED、REPEATABLE READ 和 SERIALIZABLE。通过实际操作和观察，我们深入了解了不同隔离级别如何影响并发事务的行为，并具体观察了脏读、不可重复读和幻象读等并发问题的发生机制以及各隔离级别对这些问题的防范能力。

实验环境

涉及表: TEACHERS (关键字段 tid, salary)

关键测试行: tid = '200003125', 初始 salary $\approx$ 3928 / 3978 / 4028 (实验中会调整)

并发方式: 使用两个会话 A (写) / B (读) 配合 WAITFOR DELAY 制造并发窗口

实验1: READ UNCOMMITTED (脏读)

目的

演示未提交读可以读到其他事务尚未提交、最终被回滚的数据。

过程与结果

在该实验中，事务A首先更新tid为200003125的教师工资为4200，然后通过WAITFOR DELAY延时一段时间，最后回滚事务。与此同时，事务B设置为READ UNCOMMITTED隔离级别，在事务A更新但未提交或回滚时读取salary值。从实验结果可以看到，事务B第一次读取到了4200这个并未真正提交的值，而在事务A回滚后再次读取则得到了原始值3928。这种读取到其他事务未提交数据的现象就是脏读，这也是READ UNCOMMITTED隔离级别的最大特点。

	tid	salary
1	200003125	4028

1行

事务A未提交时，事务B读取到未提交的 salary=4200（脏读）

	tid	salary
1	200003125	3928

1行

事务A回滚后，事务B再次读取恢复为 salary=3928

现象: 第一次读到 4200（脏读），回滚后恢复 3928。

脏读的存在可能导致严重的业务问题，因为基于未提交数据所做的决策可能会因为数据的回滚而变得毫无意义甚至引发错误。

实验2：READ COMMITTED（不可重复读）

目的

READ COMMITTED作为SQL Server默认的隔离级别，能够防止脏读，但共享锁在读取后立即释放，因此仍可能出现不可重复读的问题。

过程与结果

在这个实验中，事务B采用READ COMMITTED隔离级别，对同一行数据进行两次读取，中间有一定的时间间隔。在此期间，事务A会对该行数据进行更新并提交。由于READ COMMITTED级别下共享锁在读取完成后就会释放，所以事务A能够成功更新数据。当事务B再次读取相同数据时，就得到了不同的结果。实验结果显示，事务B第一次读取salary为3978，而第二次读取则变成了4028，这正是典型的不可重复读现象。

事务B 第一次读取 salary=3978
事务B 第二次读取 salary=4028

calhost [USER1] 1/6
School_Data 1/13
服务器对象
calhost [USER2] 1/6
School_Data 1/13
服务器对象
calhost [USER3] 1/6
School_Data 1/13
服务器对象

```
46  
47 -- 事务B: 提交读  
48 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
49 BEGIN TRANSACTION;  
50 DECLARE @s1 INT, @s2 INT;  
51 SELECT @s1 = salary FROM teachers WHERE tid = '200003125';  
52 PRINT '事务B 第一次读取 salary=' + CAST(@s1 AS VARCHAR);  
53  
54 WAITFOR DELAY '00:00:05'; -- 期间由事务A更新提交  
55  
56 SELECT @s2 = salary FROM teachers WHERE tid = '200003125';  
57 PRINT '事务B 第二次读取 salary=' + CAST(@s2 AS VARCHAR);  
58 COMMIT TRANSACTION;  
59  
60  
61 --在上述等待期间运行的事务A（示意，手动或另会话执行）：  
  
PRINT '事务B 第一次读取 salary=' + CAST(@s1 AS VARCHAR);  
  
WAITFOR DELAY '00:00:05'; -- 期间由事务A更新提交  
  
SELECT @s2 = salary FROM teachers WHERE tid = '200003125';  
PRINT '事务B 第二次读取 salary=' + CAST(@s2 AS VARCHAR);  
COMMIT TRANSACTION;  
PRINT '对比: 若 @s1 != @s2, 发生不可重复读（允许，因为共享锁读后释放）';  
PRINT '';  
  
事务B 第一次读取 salary=3978  
事务B 第二次读取 salary=4028
```

> 实验 > lc14 > lc14.sql

READ COMMITTED 下，两次读取结果不同，出现不可重复读

现象: 两次读取结果不同，发生不可重复读；脏读被避免。

虽然READ COMMITTED隔离级别有效避免了脏读问题，但它无法保证在同一事务内的多次读取结果一致，这对于某些需要数据一致性的应用场景来说仍然是一个隐患。

实验3：REPEATABLE READ（防不可重复读，可能幻象）

目的

验证可重复读级别能够防止同一行数据的不可重复读问题，但在此级别下仍可能出现幻象读现象。

防止不可重复读

为了验证REPEATABLE READ隔离级别对不可重复读的防范能力，我们设计了如下实验：事务B以REPEATABLE READ隔离级别对特定行进行两次读取，中间等待一段时间；与此同时，事务A尝试更新同一行数据。由于REPEATABLE READ级别会在整个事务期间持有共享锁，因此事务A的更新操作会被阻塞，直到事务B提交为止。实验结果显示，事务B两次读取的结果完全一致，证明了该隔离级别确实能够防止不可重复读问题。

School_Data 1/13

服务器对象

@localhost [USER2] 1/6

School_Data 1/13

服务器对象

@localhost [USER3] 1/6

School_Data 1/13

服务器对象

74

75 ✓

76

77

78

79

80

81

82

83

84

85

86

87

88

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
DECLARE @r1 INT, @r2 INT;
SELECT @r1 = salary FROM teachers WHERE tid = '200003125';
PRINT 'RR 第一次读取 salary=' + CAST(@r1 AS VARCHAR);

WAITFOR DELAY '00:00:10'; -- 期间：更新同一行会被阻塞；插入新的 tid 记录仍可出现

SELECT @r2 = salary FROM teachers WHERE tid = '200003125';
PRINT 'RR 第二次读取 salary=' + CAST(@r2 AS VARCHAR);
COMMIT TRANSACTION;
PRINT '若 @r1 == @r2：不可重复读被防止。仍需注意范围内插入/删除导致幻象读。';
PRINT '';
```

WAITFOR DELAY '00:00:10'; -- 期间：更新同一行会被阻塞；插入新的 tid 记录仍可出现幻象

SELECT @r2 = salary FROM teachers WHERE tid = '200003125';

PRINT 'RR 第二次读取 salary=' + CAST(@r2 AS VARCHAR);

COMMIT TRANSACTION;

RR 第一次读取 salary=4028

RR 第二次读取 salary=4028

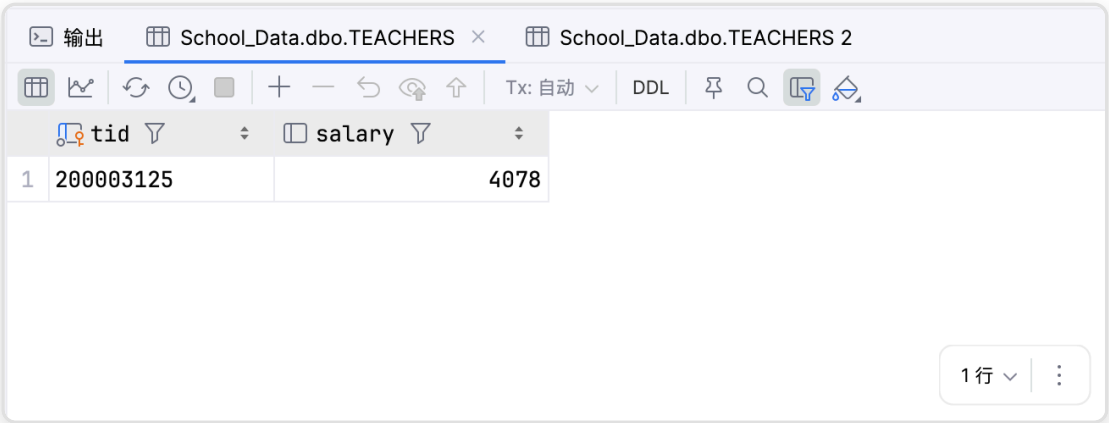
[2025-12-11 19:55:37] 在 10 s 15 ms 内完成

REPEATABLE READ 下，两次读取同一行一致，写入被阻塞

现象: 两次读取 salary 相同；写被阻塞直到事务B提交。

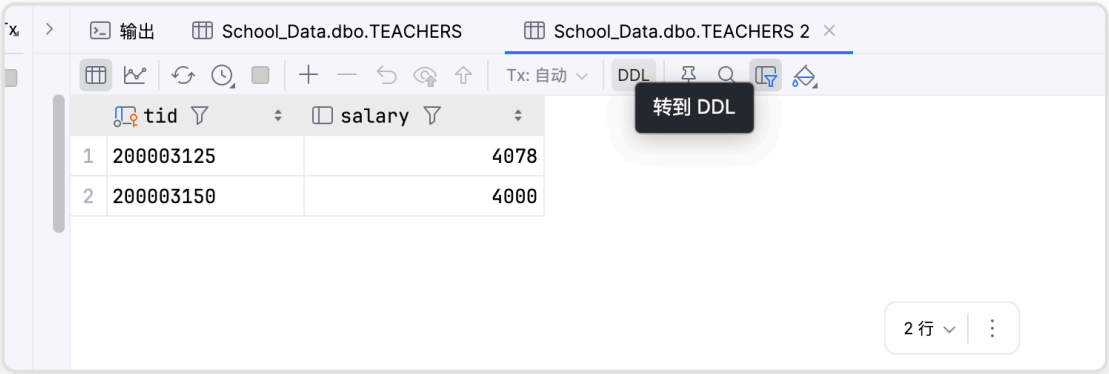
幻象读演示

尽管REPEATABLE READ能够防止不可重复读，但在处理范围查询时仍可能出现幻象读。在这一部分的实验中，事务B同样使用REPEATABLE READ隔离级别，对tid在200003100到200003200之间的教师记录进行范围查询。在两次查询之间，事务A向该范围内插入一条新的记录(tid=200003150)。结果显示，事务B的第一次查询没有返回这条新记录，而第二次查询却包含了它，这就构成了幻象读现象。这是因为在REPEATABLE READ级别下，只对已存在的记录加锁，而不会阻止新记录的插入。



	tid	salary
1	200003125	4078

第一次范围查询（无新插入行）



	tid	salary
1	200003125	4078
2	200003150	4000

第二次范围查询多出新插入的 tid=200003150，出现幻象读

现象: 第二次范围查询出现新插入的 tid=200003150，产生幻象读。

实验4：SERIALIZABLE（防幻象）

目的

验证可串行化隔离级别通过范围锁机制彻底阻止幻象读的发生。

过程与结果

在最后一个实验中，我们将事务B的隔离级别提升到最高的SERIALIZABLE级别，同样执行两次范围查询，查询tid在200003100到200003200之间的教师记录。与之前实验不同的是，SERIALIZABLE级别不仅会对已有记录加锁，还会对查询范围加锁，从而阻止任何在该范围内插入、删除或修改记录的操作。实验过程中，当事务B执行期间，事务A试图插入新记录的操作被阻塞，直到事务B完成并提交后才得以执行。因此，事务B两次查询的结果完全一致，没有出现幻象读现象。这充分证明了SERIALIZABLE隔离级别的强大保护能力。

The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query script for a transaction in SERIALIZABLE isolation level. The script includes a delay to simulate a concurrent transaction (A) attempting to insert a record during the range query. The bottom pane shows the results of the first range query, which returned three rows.

```
117 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
118 BEGIN TRANSACTION;
119
120 PRINT 'SERIALIZABLE 第一次范围查询 (tid between 200003100 and 200003200) ';
121 SELECT tid, salary FROM teachers WHERE tid BETWEEN '200003100' AND '200003200';
122
123 WAITFOR DELAY '00:00:05'; -- 期间插入/删除/更新该范围将被阻塞
124
125 PRINT 'SERIALIZABLE 第二次范围查询';
126 SELECT tid, salary FROM teachers WHERE tid BETWEEN '200003100' AND '200003200';
127 COMMIT TRANSACTION;
128 PRINT '范围锁防止幻象读，确保两次结果一致';
129 PRINT '';
```

tid	salary
200003122	4100
200003124	4100
200003125	3928

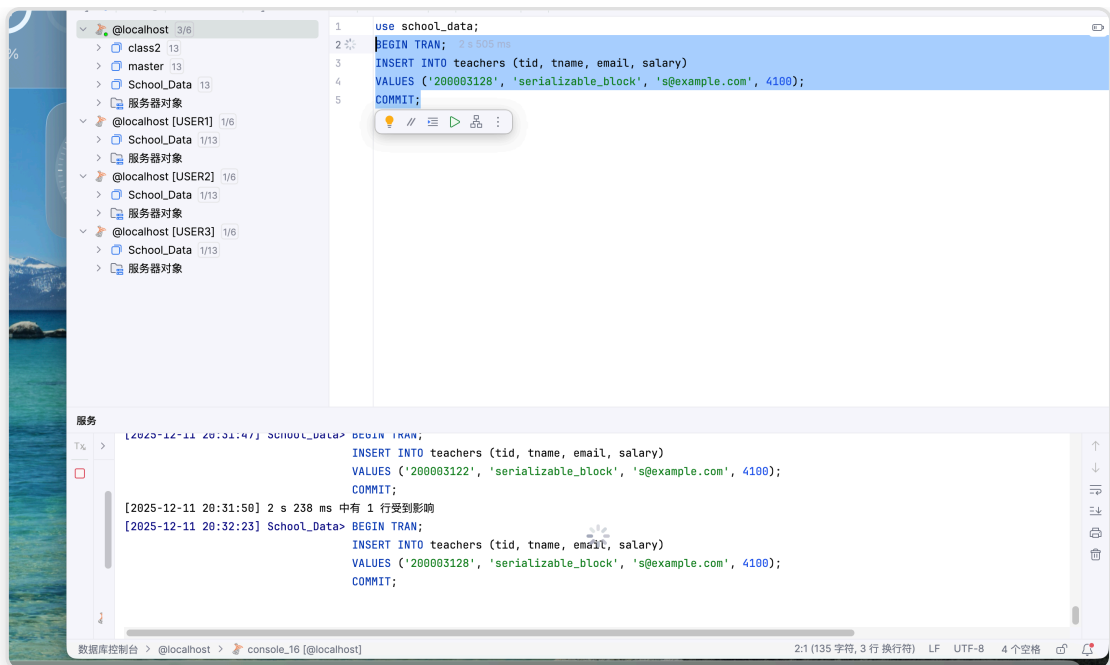
SERIALIZABLE：第一次范围查询结果

This screenshot shows the same SQL Server Enterprise Manager interface as the previous one, but with the results of the second range query. The query script is identical, but the results pane shows the same three rows as the first query, demonstrating that no phantom reads occurred because of the range lock held by the first query.

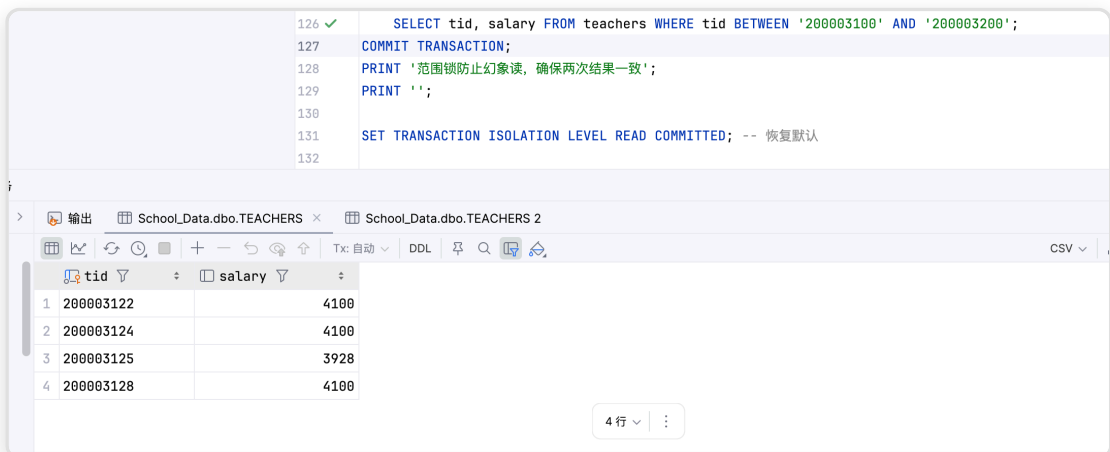
```
117 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
118 BEGIN TRANSACTION;
119
120 PRINT 'SERIALIZABLE 第一次范围查询 (tid between 200003100 and 200003200) ';
121 SELECT tid, salary FROM teachers WHERE tid BETWEEN '200003100' AND '200003200';
122
123 WAITFOR DELAY '00:00:05'; -- 期间插入/删除/更新该范围将被阻塞
124
125 PRINT 'SERIALIZABLE 第二次范围查询';
126 SELECT tid, salary FROM teachers WHERE tid BETWEEN '200003100' AND '200003200';
127 COMMIT TRANSACTION;
128 PRINT '范围锁防止幻象读，确保两次结果一致';
129 PRINT '';
```

tid	salary
200003122	4100
200003124	4100
200003125	3928

SERIALIZABLE：第二次范围查询结果，与第一次一致



事务A 在范围内的插入/更新被范围锁阻塞，等待事务B



事务B结束后，事务A才完成更新；后续读取可见更新结果

现象: 两次范围查询结果一致；插入被阻塞至事务B提交，未出现幻象。

当然，SERIALIZABLE隔离级别也并非完美无缺，它的高安全性是以牺牲并发性能为代价的，因为范围锁的存在会导致更多的阻塞情况，影响系统的整体吞吐量。

实验总结

隔离级别与并发问题的关系

通过这四个实验，我们可以清楚地看到不同隔离级别对各种并发问题的处理能力。READ UNCOMMITTED级别最为宽松，允许所有类型的并发问题发生，虽然性能最好但风险也最大；READ COMMITTED作为默认级别，能够在避免脏读的同时保持较好的并发性，但无法解决不可重复读和幻象读问题；REPEATABLE READ进一步提升了数据一致性，有效防止了不可重复读，但对幻象读仍然无能为力；SERIALIZABLE则是最高级别的保护，能够彻底杜绝所有三种并发问题，但同时也带来了最大的性能开销。

锁机制的作用

各种隔离级别的实现都依赖于不同的锁机制。共享锁主要用于读操作，在READ COMMITTED级别下读完即释放，而在REPEATABLE READ及以上级别则会持续到事务结束。排他锁用于写操作，会阻塞其他读写请求直至事务完成。而SERIALIZABLE级别更是引入了范围锁的概念，通过对查询范围加锁来防止幻象读的发生。

实践体会

这次实验让我深刻认识到数据库隔离级别选择的重要性。在实际应用中，我们需要根据业务需求在数据一致性和系统性能之间做出权衡。对于金融交易等对一致性要求极高的场景，可能需要使用SERIALIZABLE级别；而对于一些对实时性要求较高但能容忍轻微不一致的场景，则可以选择较低的隔离级别。实验中使用的WAITFOR DELAY语句和双会话配合的方式，为我们观察并发行为提供了很好的手段，有助于更好地理解数据库内部的工作机制。